

Machine Learning

Quick Recall Notes — Data Science Prep

Preprocessing · Models · Evaluation · Pipelines

1 · ML Pipeline — Standard Steps

Step	What to do	Key functions
1. Load Data	Read CSV/Excel, check shape	pd.read_csv() df.head() df.info()
2. EDA	Distributions, correlations, outliers	df.describe() sns.heatmap() boxplot
3. Clean	Handle nulls, duplicates, dtypes	fillna() dropna() drop_duplicates()
4. Feature Eng	Create/transform features	pd.get_dummies() LabelEncoder log()
5. Split	Train/test split	train_test_split(X, y, test_size=0.2)
6. Scale	Normalize/standardize numerical features	StandardScaler MinMaxScaler
7. Model	Choose algo, fit, predict	model.fit(X_train, y_train)
8. Evaluate	Check metrics on test set	accuracy, rmse, f1, r2 etc.
9. Tune	GridSearch / RandomSearch for best params	GridSearchCV RandomizedSearchCV
10. Deploy	Save model, build API	joblib.dump() pickle FastAPI

Skeleton Code — Copy This Every Project

```
import pandas as pd, numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report

# 1. Load
df = pd.read_csv('data.csv')

# 2. Features & Target
X = df.drop('target', axis=1)
y = df['target']

# 3. Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# 4. Scale
sc = StandardScaler()
X_train = sc.fit_transform(X_train) # fit+transform on train
X_test = sc.transform(X_test)       # ONLY transform on test!

# 5. Model
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# 6. Evaluate
y_pred = model.predict(X_test)
print(accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

■ ■ NEVER fit scaler on test data — only fit on train, transform both. Data leakage!

2 · Preprocessing — Encoders & Scalers

Technique	When to use	Code
StandardScaler	Most ML algos, SVM, LR	StandardScaler().fit_transform(X)
MinMaxScaler	Neural Networks (0-1 range)	MinMaxScaler().fit_transform(X)
RobustScaler	Data has outliers	RobustScaler().fit_transform(X)
LabelEncoder	Ordinal categories (low/med/high)	LabelEncoder().fit_transform(col)
OneHotEncoder	Nominal categories (no order)	pd.get_dummies(df['col'])
OrdinalEncoder	Ordered categories with custom order	OrdinalEncoder(categories=[...])
SimpleImputer	Fill missing values (in pipeline)	SimpleImputer(strategy='mean')
PolynomialFeatures	Add interaction/polynomial terms	PolynomialFeatures(degree=2)
SelectKBest	Feature selection by stats	SelectKBest(f_classif, k=10)
PCA	Dimensionality reduction	PCA(n_components=0.95)

3 · ML Algorithms — Quick Reference

Algorithm	Type	Import	When to use
LinearRegression	Regression	sklearn.linear_model	Continuous target, linear relationship
LogisticRegression	Classif.	sklearn.linear_model	Binary/multi-class, baseline
Ridge / Lasso	Regression	sklearn.linear_model	Regularized linear — many features
KNeighborsClassifier	Classif.	sklearn.neighbors	Small data, non-linear boundary
DecisionTreeClassifier	Classif.	sklearn.tree	Interpretable, fast, overfit risk
RandomForestClassifier	Classif.	sklearn.ensemble	Strong baseline, handles missing
GradientBoostingCl.	Classif.	sklearn.ensemble	Best accuracy on tabular data
XGBClassifier	Classif.	xgboost	Competitions, structured data
SVC	Classif.	sklearn.svm	High-dim, small-medium datasets
GaussianNB	Classif.	sklearn.naive_bayes	Text classification, fast
KMeans	Clustering	sklearn.cluster	Unsupervised grouping
DBSCAN	Clustering	sklearn.cluster	Finds outliers, arbitrary shapes
PCA	Dim Red.	sklearn.decomposition	Visualise, reduce features

Algorithm Code Patterns

```
# --- REGRESSION ---
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.metrics import mean_squared_error, r2_score

model = Ridge(alpha=1.0)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print(f'RMSE: {mean_squared_error(y_test, y_pred, squared=False):.2f}')
print(f'R2: {r2_score(y_test, y_pred):.3f}')
```

```
# --- RANDOM FOREST ---
from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier(n_estimators=200, max_depth=10,
                           min_samples_split=5, random_state=42)
rf.fit(X_train, y_train)
print(rf.feature_importances_) # which features matter most

# --- XGB00ST ---
from xgboost import XGBClassifier
xgb = XGBClassifier(n_estimators=300, learning_rate=0.05,
                    max_depth=6, subsample=0.8, random_state=42)
xgb.fit(X_train, y_train, eval_set=[(X_test, y_test)], verbose=False)

# --- KMEANS ---
from sklearn.cluster import KMeans
km = KMeans(n_clusters=4, random_state=42)
df['cluster'] = km.fit_predict(X)
```

4 · Evaluation Metrics — Know These Cold!

Classification Metrics

Metric	Formula / Meaning	Use when
Accuracy	Correct / Total	Balanced classes
Precision	$TP / (TP + FP)$ — of predicted +, how many right	High cost of false alarms
Recall	$TP / (TP + FN)$ — of actual +, how many found	High cost of missing positives
F1 Score	$2 * (P * R) / (P + R)$ — harmonic mean	Imbalanced classes
ROC-AUC	Area under ROC curve. >0.8 = good	Probability scores
Log Loss	Lower = better. Penalizes confident wrong preds	Probabilistic models
Confusion Matrix	TP/FP/TN/FN breakdown	Understand error types

Regression Metrics

Metric	Meaning	Code
MAE	Mean Absolute Error — avg error, same unit	<code>mean_absolute_error(y, y_pred)</code>
MSE	Mean Squared Error — penalizes large errors	<code>mean_squared_error(y, y_pred)</code>
RMSE	Root MSE — interpretable, same unit as target	<code>mean_squared_error(y, y_pred, squared=False)</code>
R2	0-1. Closer to 1 = better. Can be -ve!	<code>r2_score(y, y_pred)</code>
MAPE	% error — scale-independent comparison	<code>mean_absolute_percentage_error(y, y_pred)</code>

Metrics Code

```
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, confusion_matrix, classification_report,
    mean_absolute_error, mean_squared_error, r2_score)

# Classification
print(accuracy_score(y_test, y_pred))
print(f1_score(y_test, y_pred, average='weighted')) # multiclass
print(roc_auc_score(y_test, model.predict_proba(X_test)[: ,1]))
print(classification_report(y_test, y_pred)) # all-in-one

# Confusion matrix
import seaborn as sns
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')

# Regression
rmse = mean_squared_error(y_test, y_pred, squared=False)
r2 = r2_score(y_test, y_pred)
print(f'RMSE={rmse:.2f} R2={r2:.3f}')
```

5 · Cross Validation & Tuning

```
from sklearn.model_selection import cross_val_score, GridSearchCV, RandomizedSearchCV

# K-Fold Cross Validation
scores = cross_val_score(model, X, y, cv=5, scoring='f1_weighted')
print(f'CV F1: {scores.mean():.3f} +/- {scores.std():.3f}')

# Grid Search
```

```

params = {
    'n_estimators': [100, 200, 300],
    'max_depth':    [None, 5, 10],
    'min_samples_split': [2, 5, 10]
}
gs = GridSearchCV(RandomForestClassifier(), params, cv=5,
                  scoring='f1_weighted', n_jobs=-1, verbose=1)
gs.fit(X_train, y_train)
print(gs.best_params_)
print(gs.best_score_)

# Randomized Search (faster for large param space)
from scipy.stats import randint
params_dist = {'n_estimators': randint(100, 500), 'max_depth': randint(3, 15)}
rs = RandomizedSearchCV(rf, params_dist, n_iter=30, cv=5, random_state=42)
rs.fit(X_train, y_train)

```

■ *Always use cross_val_score before trusting a single train/test split result!*

6 · sklearn Pipeline — Production Ready

```

from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestClassifier

# Separate numeric and categorical columns
num_cols = ['age', 'salary', 'experience']
cat_cols = ['city', 'department']

# Numeric pipeline
num_pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

# Categorical pipeline
cat_pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OneHotEncoder(handle_unknown='ignore'))
])

# Combine
preprocessor = ColumnTransformer([
    ('num', num_pipe, num_cols),
    ('cat', cat_pipe, cat_cols)
])

# Full pipeline with model
full_pipe = Pipeline([
    ('prep', preprocessor),
    ('model', RandomForestClassifier(random_state=42))
])

full_pipe.fit(X_train, y_train)
y_pred = full_pipe.predict(X_test)

```

■ *Pipeline prevents data leakage — preprocessing happens inside CV folds automatically!*

7 · Save, Load & Interview Quick Tips

```

import joblib

# Save
joblib.dump(full_pipe, 'model.pkl')

```

```
# Load & predict on new data
model = joblib.load('model.pkl')
prediction = model.predict(new_data)

# Feature importance (tree-based models)
importances = model['model'].feature_importances_
feature_names = num_cols + list(model['prep']
    .named_transformers_['cat']['encoder']
    .get_feature_names_out(cat_cols))
pd.Series(importances, index=feature_names).sort_values().plot.barh()
```

Interview Question — Cheat Sheet

- **Bias-Variance Tradeoff:** High bias = underfitting (simple model) | High variance = overfitting (complex). Balance with regularization + cross-validation.
- **Overfitting fixes:** More data, dropout (DL), regularization (L1/L2), max_depth, pruning, cross-validation.
- **Imbalanced classes:** Use class_weight='balanced', SMOTE, precision/recall instead of accuracy.
- **Feature importance:** Use RandomForest .feature_importances_, SHAP, or correlation with target.
- **When to use what:** Linear data → LR | Non-linear → RF/XGB | Text → NB/LR | Small data → SVM | Clustering → KMeans.
- **Data leakage:** When test data info leaks into training. Always fit preprocessors only on train set!
- **Why normalize?** Distance-based (KNN, SVM) and gradient-based (LR, NN) algos need scaled features. Tree models don't.
- **Cross-validation:** Better than single split — reduces variance in evaluation. Use 5-fold or 10-fold.
- **GridSearch vs RandomSearch:** Grid = exhaustive but slow | Random = faster, good for large param spaces.
- **R2 score:** Can be negative! Means model worse than predicting mean. Target > 0.7 for decent model.

Revision Strategy: Cover 'Code' column → write from memory → run it → debug → repeat. Do 1 project cycle weekly.